本科毕业论文

基于 FastAPI 与 Vue 的校园二手商品交易平台设计与实现

Design and Implementation of a Campus Second-hand Goods Trading Platform Based on FastAPI and Vue

学生姓名 _____

学号 _____

专业 _____

指导教师 _____

学院 _____

完成日期 2026 年 8 月

目录

打开 Word 后按 Ctrl+A，再按 F9 更新目录和页码。

图目录

打开 Word 后更新域生成图目录。

表目录

打开 Word 后更新域生成表目录。

摘要

高校学生在课程更替、宿舍搬迁、电子设备更新和毕业离校等阶段会持续产生教材、数码产品、生活用品与文体器材等闲置物品。传统交易通常依赖微信群、QQ群、表白墙或阶段性跳蚤市场，存在信息分散、检索困难、商品字段不统一、交易身份难确认、沟通过程缺少上下文、订单履约不可追踪以及违规内容依赖人工发现等问题。为提高校园闲置资源的流转效率，并为平台运营提供可审核、可统计和可追溯的管理能力，本文设计并实现了一套面向校园场景的二手商品交易平台。

系统采用 B/S 架构与前后端分离模式。前端使用 Vue 3、Vue Router、Pinia、Axios、Element Plus 和 ECharts，实现面向学生用户的 C 端与面向运营人员的管理后台；后端使用 Python、FastAPI、Pydantic 与 SQLAlchemy 构建 REST API，演示环境采用 SQLite 保存业务数据，并使用 PBKDF2-HMAC-SHA256 完成密码哈希、使用 HMAC-SHA256 签名访问令牌完成身份识别。系统围绕用户、分类、商品、收藏、订单、会话、消息、评价、举报、通知、轮播、公告、审计日志和系统配置 14 类核心实体组织数据，实现商品浏览检索、发布审核、收藏沟通、订单状态流转、交易评价、通知举报以及后台用户管理、商品审核、订单查询、举报处理、运营内容维护、系统配置和数据可视化等功能。

本文按照需求分析、总体设计、详细实现和系统测试的过程开展研究。测试结果表明，前端生产构建与后端语法检查能够通过，主要页面可在桌面端和移动端正常使用；在本机演示环境中，对 4 个只读接口分别发起 30 次请求、并发数为 5 时，全部 120 次请求均成功，平均响应时间为 4.01 ~ 15.26 ms，P95 响应时间为 4.79 ~ 23.77 ms。该结果仅用于验证原型的基本响应能力，不代表生产环境容量。当前系

统能够形成从商品发布、后台审核、买卖沟通、创建订单、线下面交到评价与风控的业务闭环，可为校园二手交易类课程设计与毕业设计提供完整的工程实现参考。

关键词：校园二手交易；前后端分离；FastAPI；Vue 3；信息管理系统

Abstract

University students continuously generate idle textbooks, digital devices, household items, and sports or entertainment goods during course changes, dormitory relocation, equipment upgrades, and graduation. Traditional campus transactions often rely on social media groups, campus confession walls, or temporary flea markets. These channels suffer from fragmented information, weak searchability, inconsistent product descriptions, uncertain identities, context-free communication, untraceable fulfillment, and limited content governance. To improve the circulation efficiency of idle resources and provide auditable, measurable, and traceable operational capabilities, this thesis designs and implements a campus second-hand goods trading platform.

The system adopts a browser/server architecture and a front-end/back-end separation pattern. Vue 3, Vue Router, Pinia, Axios, Element Plus, and ECharts are used to implement both the student-facing client and the administration console. The back end is built with Python, FastAPI, Pydantic, and SQLAlchemy through REST APIs. SQLite is used for local demonstration data. Passwords are protected with PBKDF2-HMAC-SHA256, while HMAC-SHA256 signed access tokens are used for authentication. The data model covers fourteen core entities, including users, categories, products, favorites, orders, conversations, messages, reviews, reports, notifications, banners, announcements, audit logs, and system configurations. The platform implements product discovery, publishing and moderation, favorites and messaging, order state transitions, reviews, notifications, reports, user administration, product auditing, order querying, risk handling, content operations, system settings, and visual analytics.

The research follows the process of requirement analysis, architectural design, implementation, and system testing. The front-end production build and back-end syntax checks passed, and the main pages worked on both desktop and mobile layouts. In a local demonstration benchmark, four read-only endpoints each received thirty requests at a concurrency level of five. All 120 requests succeeded. Mean response times ranged from 4.01 to 15.26 ms, and P95 response times ranged from 4.79 to 23.77 ms. These

measurements only demonstrate the responsiveness of the local prototype and must not be interpreted as production capacity. The implemented system establishes a business loop from product publication and moderation to communication, ordering, face-to-face delivery, review, and risk control, providing an engineering reference for campus trading information systems.

Keywords: campus second-hand trading; front-end and back-end separation; FastAPI; Vue 3; information management system

第 1 章 绪论

1.1 研究背景

随着高校数字化服务不断深入，学生的学习、生活与消费活动逐渐从线下转向网络平台。教材更新、宿舍搬迁、兴趣变化、电子产品升级和毕业离校会产生大量仍具有使用价值的闲置物品。二手流转能够降低学生购买学习和生活用品的成本，也能延长商品使用周期，减少可复用物品被直接丢弃造成的资源浪费。教育部发布的绿色低碳发展国民教育体系建设实施方案强调将绿色低碳理念融入教育体系并倡导简约适度、绿色低碳的生活方式[1]，因此，构建方便、安全和可持续的校园闲置交易渠道具有现实意义。

与一般社会化二手市场相比，校园交易具有范围相对封闭、用户身份相近、商品单价整体较低、线下面交比例较高和需求时效性强等特点。教材需要与课程和学期匹配，自行车、小家电等商品不便长距离邮寄，学生也更倾向在图书馆、食堂、宿舍区等校内公共场所完成验货和交接。校园内部天然的地理接近性可以降低物流成本，但也对身份确认、见面地点、履约记录和纠纷处理提出了更具体的要求。

目前，校内闲置交易仍大量依赖微信群、QQ 群、表白墙或朋友圈。此类渠道虽然使用门槛低，却缺少统一的商品标题、分类、成色、价格和取货地点字段，旧消息会快速被新消息覆盖，用户难以按关键词、价格或分类筛选。买卖双方从商品信

息转移到私聊后，沟通内容与交易对象容易脱节；一旦发生爽约、描述不符或违规商品问题，也缺少订单、举报和处理记录。阶段性跳蚤市场又受到时间和地点限制，不能满足日常持续交易需求。

相关校园二手交易研究指出，闲置资源流转对资源共享和绿色消费具有积极作用，但平台仍需要拓宽信息渠道、完善交易保障和运营机制[2-4]。因此，本文以校园交易的真实过程为对象，设计一个同时覆盖学生端和管理端的平台，将商品发布、审核、搜索、沟通、订单、评价、举报和运营数据纳入统一信息系统。

1.2 国内外研究现状

1.2.1 国外研究现状

国外二手交易与共享经济研究较早关注可持续消费、用户信任、在线声誉以及 C2C 平台的交易成本。面向高校的实践研究通常将资源再利用、节约消费与学生群体的集中性结合起来。Wei 等对 663 名大学生开展调查，结果显示教材、电子设备和交通工具是具有明显二手需求的类别，受访者同时高度关注商品质量与使用情况[3]。这表明校园平台不能只提供信息发布，还需要结构化商品描述、评价记录和风险提示。

在系统技术方面，现代 Web 平台普遍采用客户端—服务器架构和资源化接口。Fielding 在网络应用架构研究中提出 REST 架构风格，强调统一接口、无状态交互和组件独立演进[5]。此类设计有利于浏览器端、移动端和后台管理端共享服务能力。国外商业平台还常使用实时消息、推荐算法、支付托管和信用风控，但这些能力通常依赖成熟的身份、支付与数据基础设施，并不能直接等同于高校原型系统的实现范围。

1.2.2 国内研究现状

国内校园二手交易研究主要集中在需求调查、平台模式和信息系统实现三个方向。市场研究认为大学生闲置物品具有数量多、品类集中和交易半径小等特征，信息不对称、渠道不成熟和缺少保障会阻碍交易效率[2][4][6]。部分平台实践引入信用评价、隐私保护和校园认证，试图利用校园半熟人社会降低信任成本。信息系统类研究则多采用 Java Web、Spring Boot 或微信公众平台实现商品发布、分类浏览和用户管理等功能[7-8]。

现有研究为校园平台建设提供了功能参考，但部分系统仍偏重页面展示，缺少商品审核、订单状态、消息上下文、举报处理和管理审计之间的关联；部分论文对性能和安全性的结论缺少测试环境与原始数据；还有的系统仅面向用户端，没有将运营治理作为完整子系统。随着前后端分离开发模式普及，使用现代 Python API 框架和 Vue 组件化前端实现双端协同，具有较强的工程实践价值。

1.2.3 现有研究不足与本文切入点

综合已有研究，校园二手交易系统主要存在四方面改进空间。第一，功能链条不完整，商品发布、沟通、订单与评价常被设计为相互独立的页面。第二，平台治理薄弱，商品审核、举报处理、用户状态和操作日志没有形成闭环。第三，校园特征不足，缺少校区、校内取货地点、学生认证状态和面交安全提示。第四，研究证据不足，系统截图、数据库模型、接口实现与测试用例之间缺少对应关系。

本文的切入点是以当前可运行系统为研究对象，采用 Vue 3 和 FastAPI 构建 C 端与管理后台，将 14 类核心实体和 49 个接口映射到完整业务流程；在论文中同时给出需求、架构、数据库、状态机、真实页面截图和可复现测试结果，并明确演示级认证、模拟付款、REST 消息与 SQLite 数据库的边界。

1.3 研究目的与意义

1.3.1 研究目的

本文旨在设计并实现一套面向高校学生的二手商品交易平台。系统需要实现用户注册登录、商品浏览筛选、商品发布审核、收藏、商品会话、订单状态流转、交易评价、通知与举报，同时为管理员提供数据概览、用户管理、商品审核、订单查询、举报风控、分类轮播公告和系统配置等能力。通过统一数据模型与接口约定，使需求、设计、实现和测试能够相互追溯。

1.3.2 理论意义

本文将软件工程中的角色分析、业务流程、模块划分、状态机、数据建模和接口设计应用于校园二手交易场景，分析 C2C 交易与校园运营治理之间的关系。研究不以提出新算法为目标，而是验证完整信息系统方法在小范围、线下面交型交易中的适用性，为同类校园服务系统的工程建模提供案例。

1.3.3 实践意义

对学生用户而言，平台能够集中展示同校闲置商品，降低检索和沟通成本；订单、评价和举报记录能够提高交易过程的可追溯性。对运营人员而言，后台能够统一处理用户、商品、订单和风险事件，并通过统计图表了解平台运行状态。对教学实践而言，系统涵盖前端、后端、数据库、安全、测试和文档，可作为前后端分离项目的完整实现样例。

1.4 研究内容与研究方法

1.4.1 主要研究内容

本文研究内容包括五个方面：一是分析校园闲置交易的用户角色、业务流程和功能需求；二是确定前后端分离架构、技术栈与安全边界；三是设计 C 端、管理后台、数据实体、状态机和 REST API；四是实现商品、沟通、订单、评价、举报与运营管理等功能；五是通过构建检查、页面验证、功能用例和小规模接口响应测试评价系统。

1.4.2 研究方法

本文采用文献研究法分析校园二手交易与 Web 信息系统相关成果；采用需求分析法梳理游客、校园用户和管理员的目标；采用面向对象与数据库建模方法划分实体和关系；采用原型迭代法完成页面与接口实现；采用黑盒测试和代码级检查验证主要功能，并对测试结果的适用范围进行限制说明。

1.4.3 技术路线

研究首先从场景问题提取需求，随后完成角色、用例和业务流程分析；在此基础上进行系统架构、功能模块、数据库、状态机和接口设计；然后使用 Vue 3 与 FastAPI 完成双端实现；最后通过前端构建、后端检查、浏览器验证和只读接口测试形成证据。该路线遵循“问题—需求—设计—实现—测试—总结”的工程逻辑。

1.5 论文组织结构

全文共七章。第 1 章介绍研究背景、现状、目的与方法；第 2 章介绍系统采用的架构、前端、后端、数据和安全技术；第 3 章分析角色、流程、功能与非功能需求；第 4 章完成总体架构、模块、状态、数据库、接口和安全设计；第 5 章说明关

键模块实现并展示系统页面；第 6 章给出功能、接口、权限、性能和兼容性测试；第 7 章总结研究成果、局限和改进方向。

1.6 本章小结

本章从校园闲置物品流转的现实需求出发，说明传统社交渠道在信息组织、过程留痕和平台治理方面的不足，明确了以双端协同和完整业务闭环为重点的研究方向，并给出了论文的研究内容、方法和章节结构。

第 2 章 系统相关技术

2.1 B/S 架构与前后端分离模式

B/S 架构以浏览器作为主要客户端，用户无需安装专用桌面程序即可访问系统。校园平台包含学生 C 端和管理后台，两者都通过浏览器运行，因此 B/S 架构能够降低使用和维护成本。前后端分离进一步将页面渲染与业务服务拆分：Vue 负责组件、路由、状态和交互，FastAPI 负责权限、业务规则和数据读写，双方通过 JSON 接口通信。

这种划分使前端可独立组织响应式页面，后端可统一向多个界面提供数据服务。系统中的 /api 路径代表服务接口，/uploads 提供上传图片，前端开发服务器通过代理访问后端，从而避免在组件中直接拼接不同环境地址。分离模式也要求后端不能信任前端提交的角色或状态，关键权限和业务转换必须在服务端再次验证。

2.2 Vue 3 前端框架

Vue 是用于构建用户界面的渐进式框架，可通过组件化方式组织复杂单页应用[9]。本系统使用 Vue 3 单文件组件，将页面结构、逻辑和样式按视图组织。HomeView、MarketView、ProductDetailView、OrdersView 等视图负责业务页面，ProductCard 等组件负责重复商品信息展示，CLayout 与 AdminLayout 分别提供两端的导航和页面框架。

Vue 的响应式数据机制适合展示接口返回的商品列表、订单状态和后台统计数据。Composition API 与 <script setup> 语法使加载、筛选、提交和弹窗状态能够集中在组件逻辑中。系统还通过异步组件进行路由级加载，使 C 端和管理后台页面在需要访问时再加载相应代码。

2.3 Vue Router、Pinia 与 Axios

Vue Router 是 Vue 官方路由方案，支持嵌套路由、动态参数和导航控制[10]。本系统的 C 端使用根布局嵌套首页、市场、商品详情、发布、收藏、消息、订单和个人中心；管理后台使用 /admin 布局嵌套数据概览、用户、商品、订单、举报、运营和设置页面。全局导航守卫根据 requiresAuth 和 requiresAdmin 元信息判断访问权限。

Pinia 用于跨组件共享登录令牌和用户信息。登录后，状态被同步到浏览器本地存储，刷新页面时可以恢复。Pinia 官方将其定位为 Vue 的状态管理方案，并提供简洁的 Store、Getter 和 Action 组织方式[11]。Axios 负责 HTTP 请求；请求拦截器自动在请求头加入 Bearer 令牌，响应拦截器统一提取后端错误信息，并在收到 401 响应时清理失效登录状态。

2.4 Element Plus 与 ECharts

Element Plus 为表单、按钮、选择器、对话框、表格、分页和消息提示提供统一组件，能够降低管理后台和复杂表单的实现成本。商品发布页使用输入框、选择器、上传和确认控件，后台商品、用户与举报页面使用表格、筛选和弹窗完成操作。系统通过自定义校园绿视觉变量保持 C 端与后台的一致性。

ECharts 用于管理后台的数据可视化。数据概览页展示近 7 日订单与交易额趋势以及商品分类分布，图表数据来自后端 /api/admin/dashboard 接口。可视化将用户数、商品数、订单数和交易额等摘要指标与时间趋势结合，帮助管理员了解平台近期运行情况。

2.5 FastAPI Web 框架

FastAPI 是基于 Python 类型提示构建 API 的 Web 框架，支持 OpenAPI、JSON Schema、自动接口文档、参数校验和依赖注入[12]。本系统将每个业务能力声明为路径操作，例如用户登录、商品查询、订单处理和后台审核。Pydantic 模型定义请求体字段及长度、数值范围和枚举动作，能够在进入业务逻辑前完成基础数据校验。

依赖注入用于提供数据库会话、当前用户和管理员用户。get_current_user 负责要求登录，get_admin_user 在此基础上验证角色，使管理接口不依赖前端页面的可见性。FastAPI 生命周期函数在应用启动时建表并写入演示数据，静态文件挂载用于提供上传图片。

2.6 SQLAlchemy ORM 与 SQLite 数据库

SQLAlchemy ORM 将 Python 类映射为关系数据库表，并通过 Session 完成查询与持久化[13]。本系统在 models.py 中声明 14 个实体，在接口中使用 select、where、joinedload、聚合函数和 Session 提交完成数据读写。对象关系映射能够减少手工拼接 SQL 的数量，并使用户、商品、分类、订单和消息之间的关系更清晰。

SQLite 是嵌入式、无独立服务器进程的数据库，完整数据库保存在单个文件中，具有零配置和便于演示的特点[14]。因此本文在本地原型中使用 SQLite，使项目启动后可自动建表与生成数据。但是 SQLite 并非面向所有高并发服务场景的替代方案，官方文档也指出服务器型数据库能够提供更精细的并发控制[15]。生产部署应迁移到 PostgreSQL 或 MySQL，并补充事务锁、备份和监控。

2.7 RESTful API 设计

REST 将业务对象抽象为资源，并通过 URI、HTTP 方法和表示完成交互[5]。系统使用 GET 查询资源，POST 创建资源或执行非幂等业务，PUT 更新完整业务对象，PATCH 修改状态，DELETE 表示商品下架等操作。接口统一使用 /api 前缀，商品资源使用 /api/products，订单资源使用 /api/orders，后台资源使用 /api/admin 前缀。

接口返回 JSON 数据，异常通过标准 HTTP 状态码和 detail 字段表达。例如未登录返回 401、无管理员权限返回 403、资源不存在返回 404、重复进行中订单返回 409。统一约定有利于前端拦截器集中处理错误，也使 Swagger UI 能够根据 OpenAPI 自动展示请求结构。

2.8 PBKDF2 密码哈希与令牌认证

密码不能以明文保存。系统为每个密码生成 16 字节随机盐，使用 PBKDF2-HMAC-SHA256 迭代 160000 次生成派生值，并将算法、迭代次数、盐和值组合保存。登录校验时重新计算并使用恒定时间比较函数，降低直接比较可能产生的时序风险。

身份令牌由头部、载荷和 HMAC-SHA256 签名组成，载荷包含用户编号、角色、签发时间和过期时间。后端接收 Bearer 令牌后验证签名与过期时间，再从数据库获取用户并检查账号状态。当前实现适合本地演示；生产环境必须更换默认密钥，并进一步增加刷新令牌、会话撤销、登录限流、HTTPS 和密钥轮换。

2.9 本章小结

本章介绍了系统使用的前后端分离架构，以及 Vue 3、Vue Router、Pinia、Axios、Element Plus、ECharts、FastAPI、SQLAlchemy、SQLite、REST、密码哈希

和令牌认证等技术。各项技术均与后续实现存在直接对应关系，为系统设计与开发提供基础。

第 3 章 系统需求分析

3.1 系统建设目标

系统建设目标是为同校学生提供结构化、可检索、可沟通和可追踪的闲置商品交易服务，同时为管理员提供内容治理与运营管理工具。C 端应覆盖从发现商品到交易评价的主要过程，管理端应覆盖用户、商品、订单、举报、运营内容和配置。系统还应保持清晰的权限边界、统一的数据模型和可维护的前后端结构。

3.2 系统可行性分析

3.2.1 技术可行性

Vue 3 与 FastAPI 均具有成熟文档和生态，能够分别承担复杂页面和 REST 服务。SQLAlchemy 支持关系映射与多种数据库方言，演示阶段可使用 SQLite，后续可迁移到服务器型数据库。项目已实际完成前端构建和后端运行，说明技术组合能够支撑当前功能规模。

3.2.2 经济可行性

系统所用核心框架均为开源软件，本地开发不需要商业许可证。SQLite 无需单独安装数据库服务，图片使用项目本地资源，降低了原型搭建成本。真实上线时仍需考虑服务器、对象存储、域名证书、短信或邮件认证和运维监控费用，但不影响毕业设计阶段的经济可行性。

3.2.3 操作可行性

学生用户熟悉搜索、商品卡片、收藏、消息和订单等电商交互。系统顶部导航提供首页、逛市场、消息、订单和发布入口，个人头像菜单提供收藏与资料入口。

后台使用侧边导航和表格筛选，符合常见管理系统操作习惯。桌面端与移动端响应式布局降低了终端限制。

3.2.4 合规与安全可行性

系统不处理真实资金，当前付款仅为状态模拟；校园认证也没有连接学校权威身份系统。原型避免采集不必要的敏感信息，并通过密码哈希、令牌验证和角色权限保护基本账户安全。若进入真实运营，需要补充隐私政策、用户协议、认证材料管理、违规商品规则、文件安全、日志脱敏和数据保留策略。

3.3 系统角色分析

3.3.1 游客角色

游客可以访问首页、商品市场和商品详情，查看分类、价格、成色、校区、卖家公开信息和安全提示。游客不能收藏、发布、发消息、创建订单或举报；触发受保护操作时应跳转登录页，并在登录后返回原目标页面。

3.3.2 校园用户角色

校园用户在游客能力基础上，可以维护个人资料和校园信息，发布与管理商品，收藏商品，围绕商品建立会话，创建订单并根据身份处理订单状态，完成交易后评价对方，接收通知并提交举报。用户既可能是买家，也可能是卖家，订单页面应支持按买入和卖出角色筛选。

3.3.3 管理员角色

管理员可以访问管理后台，查看用户、商品、订单、交易额和举报指标，管理用户状态，审核商品，查询订单，处理举报，维护分类、轮播和公告，修改系统参

数并查看审计日志。管理员不应通过普通前端变量直接获得权限，所有后台接口都需要服务端角色校验。

系统总体用例关系如图 3-1 所示。

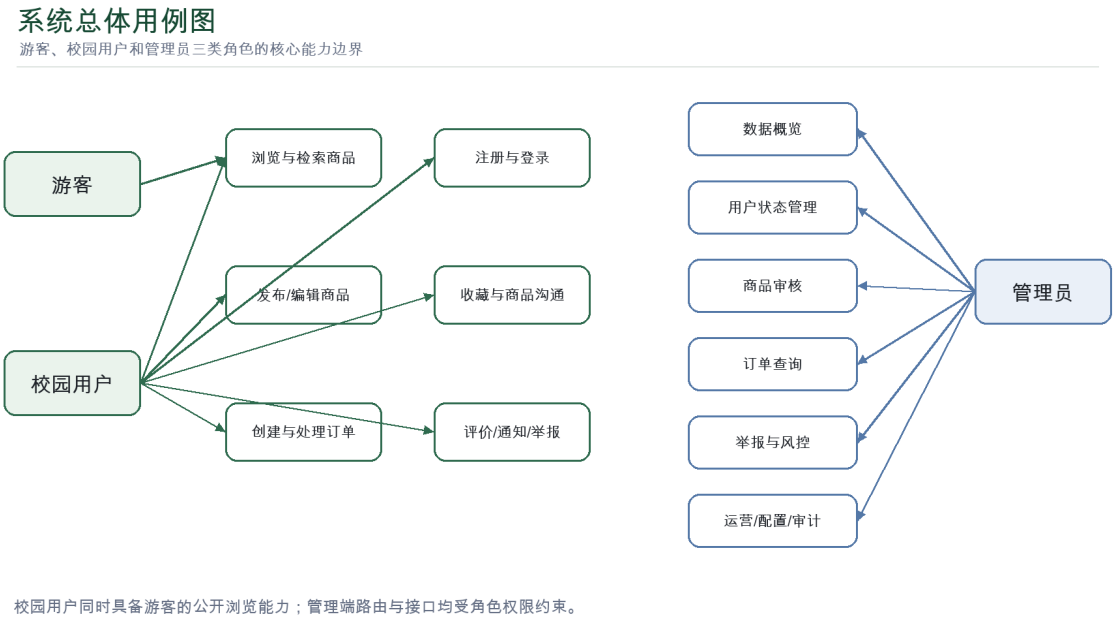


图 3-1 系统总体用例图

3.4 系统业务流程分析

3.4.1 用户注册与登录流程

注册时用户提交账号、邮箱、密码、昵称、学校和可选学号；后端检查账号和邮箱唯一性，将密码哈希后保存，并返回令牌与用户信息。登录时可使用用户名、邮箱或手机号，后端校验密码和账号状态。前端保存令牌，Axios 自动附加认证头，路由守卫根据登录与角色状态控制页面访问。

3.4.2 商品发布与审核流程

用户登录后进入发布页，上传最多 9 张图片，填写标题、分类、描述、价格、成色、校区、取货地点和交易方式。后端依据 audit_required 配置决定普通用户的新

商品是否进入 pending。管理员可审核通过、驳回、下架、恢复或设置推荐；普通用户修改商品后重新进入待审核状态。

3.4.3 商品浏览与沟通流程

用户可从首页分类或市场页进入商品列表，按关键词、分类、成色、校区和价格筛选，并按推荐、最新、价格或热度排序。进入详情页后，系统增加浏览量，展示卖家、取货位置和同类商品。登录用户可以收藏，或者以当前商品为上下文创建唯一会话并发送消息。

3.4.4 下单与面交流程

买家在商品详情页确认见面地点与备注后创建订单。系统生成订单编号和取货码，并通知卖家。买家可执行付款，交易双方可确认面交，买家完成订单后商品状态变为已售。进行中订单还可进入取消、退款或争议分支。由于当前系统未接入真实支付和取货码核销，付款与面交状态仅代表平台业务记录。

3.4.5 举报与处理流程

用户可针对商品、用户或订单提交举报原因与描述。举报初始状态为待处理，管理员可以更新为跟进或解决，并填写处理结论。完成处理时系统记录处理人和时间，并向举报人创建结果通知。关键操作写入审计日志，以便回溯。

系统的端到端核心流程如图 3-2 所示。

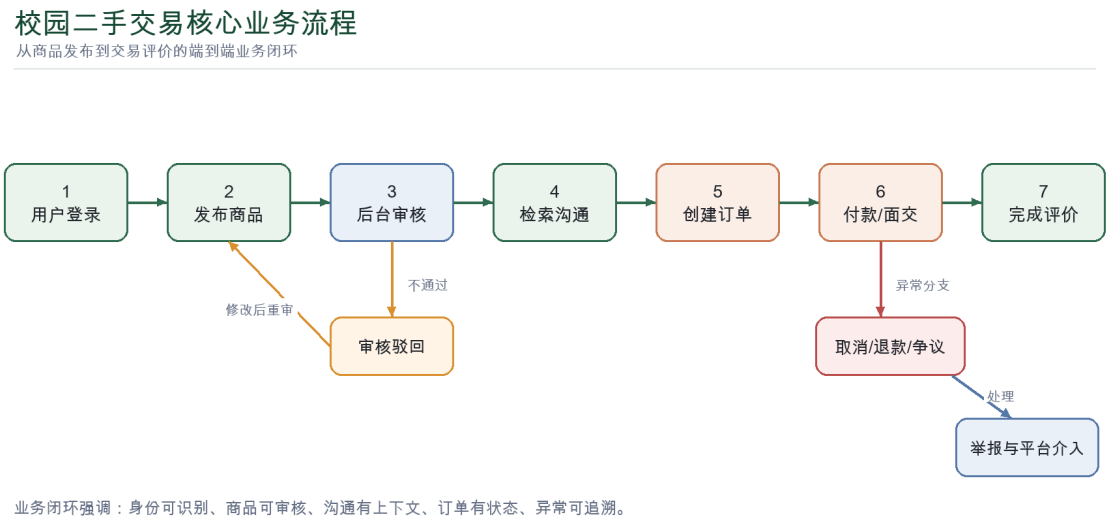


图 3-2 校园二手交易核心业务流程

3.5 C 端功能需求

3.5.1 账户与个人中心需求

系统应支持注册、登录、退出、登录状态恢复与资料编辑。个人中心需要展示昵称、头像、学校、学号、认证状态、信用分、个人简介、我的发布与通知。用户能够查看商品审核状态、重新编辑或下架自己的商品，并将通知批量标记为已读。

3.5.2 首页与商品检索需求

首页需要展示校园主题、商品分类、推荐商品、安全交易提示和平台特色。市场页需要提供关键词、分类、成色、校区、最低价、最高价与排序条件，并支持分页。商品卡片应显示图片、标题、成色、地点、价格、原价、卖家和收藏状态。

3.5.3 商品发布与管理需求

发布表单应限制标题、描述、价格与图片数量，要求用户选择有效分类并填写取货地点。用户修改商品后应重新进入审核流程，避免已发布内容绕过审核。商品

存在待审核、已发布、已驳回、已下架与已售等状态，不同状态应显示相应提示和操作。

3.5.4 收藏与消息需求

收藏操作应对同一用户和商品保持唯一关系，并同步商品收藏数量。消息应围绕具体商品建立会话，同一商品、买家和卖家组合只保留一个会话。消息需要记录发送者、内容、类型、时间和已读状态；当前通过 REST 接口加载与发送，不宣称 WebSocket 实时推送。

3.5.5 订单与评价需求

订单应记录商品、买家、卖家、金额、状态、取货码、见面地点、备注和关键时间。系统应根据买家或卖家身份限制状态动作，并禁止用户购买自己的商品。完成订单后允许双方各评价一次，评价包含 1～5 分和文字内容，用于展示用户信用。

3.5.6 通知与举报需求

订单创建、举报处理等事件应生成站内通知。用户可以查看通知并标记已读。举报应记录对象类型、对象编号、原因、描述、状态、处理人和结果，使异常交易能够进入管理流程。

3.6 管理后台功能需求

管理后台包括七类需求。数据概览需要展示平台用户、累计商品、累计订单、交易金额、待审核和待处理举报，并提供近 7 日趋势与分类分布。用户管理需要支持搜索、角色和状态筛选，以及暂停、恢复或封禁。商品管理需要审核、驳回、下架、恢复和推荐。订单管理需要按编号、商品或用户查询并查看状态。举报与风控需要按状态和对象类型筛选并填写处理结论。运营管理需要维护分类、轮播与公

告。系统设置需要保存审核开关、图片数量、服务费等参数，并展示管理员审计日志。

3.7 非功能性需求

3.7.1 性能需求

在课程设计和本地演示场景下，首页、分类和商品列表等常用只读接口应在正常网络与数据规模下快速返回，页面操作不应出现明显阻塞。列表接口必须支持分页，避免一次返回全部数据。性能结论需要给出硬件、请求数量、并发数和统计口径，不能以本机结果推断生产容量。

3.7.2 安全需求

密码必须哈希保存，接口必须验证令牌、用户状态和角色；商品修改、订单动作和后台操作要校验资源归属。上传文件需要限制扩展名，生产环境还需限制大小与 MIME、清理 EXIF 并进行内容检测。默认密钥不得用于生产，敏感接口需要 HTTPS、限流、日志脱敏和会话撤销。

3.7.3 易用性需求

页面应具有一致的导航、按钮层级、空状态、加载状态与错误提示。商品信息字段应采用学生易理解的词汇，订单状态应配合颜色和操作说明。发布页应提供图片与定价提示，交易页面应持续展示校内公共场所面交和贵重物品验货提醒。

3.7.4 兼容性与响应式需求

系统应支持现代桌面浏览器与常见手机宽度。桌面端导航、商品网格和后台表格在窄屏时需要换行或重排，页面不得产生横向整体溢出。图片应保持比例，表单控件和操作按钮应具有足够的点击区域。

3.7.5 可维护性与可扩展性需求

前端按布局、组件、视图、路由、状态和请求层组织，后端按模型、序列化、安全、数据库和路由组织。接口路径、状态值和响应结构需保持统一。数据库使用 ORM 降低方言耦合，为迁移 PostgreSQL/MySQL、增加 WebSocket、认证审核和售后工单预留扩展空间。

3.8 本章小结

本章确定了系统的建设目标与可行性，分析了游客、校园用户和管理员三类角色，描述了登录、发布审核、检索沟通、订单面交和举报处理流程，并从 C 端、管理后台和非功能层面给出了完整需求。

第 4 章 系统总体设计

4.1 设计目标与原则

系统设计遵循业务闭环、前后端分离、服务端可信、状态明确、数据可追溯和渐进扩展原则。业务闭环要求每项核心功能能够从用户操作追踪到接口与数据；服务端可信要求权限和状态转换不依赖前端隐藏按钮；状态明确要求商品、订单、举报和用户状态使用有限集合；数据可追溯要求管理操作写入审计日志；渐进扩展要求本地 SQLite 与 REST 原型可以在后续替换或增强。

4.2 系统总体架构设计

系统分为客户端表现层、前端应用层、接口与业务层以及数据与资源层，如图 4-1 所示。



图 4-1 系统总体架构图

4.2.1 前端表现层

表现层包括 C 端和管理后台。C 端强调商品发现、沟通与交易，采用顶部导航、校园主题与商品卡片；后台强调数据密度和批量管理，采用侧边栏、统计卡片、图表、表格和弹窗。两端共享颜色、字体和组件规范，但通过不同布局区分用户目标。

4.2.2 API 与业务处理层

FastAPI 接收前端请求，根据路由进入具体业务函数。Pydantic 校验请求数据，依赖函数解析当前用户和管理员角色，业务逻辑检查商品状态、资源归属和订单动作，最终通过序列化函数生成 JSON。异常以 HTTP 状态码返回，由前端统一提示。

4.2.3 数据持久层

数据持久层由 SQLAlchemy 模型、Session 和 SQLite 构成。模型声明字段、外键、唯一约束与关系；Session 管理查询、修改与提交；SQLite 保存原型数据。joinedload 用于在商品和订单查询中提前加载关联对象，减少序列化阶段的重复查询。

4.2.4 静态资源与上传文件层

前端公共目录保存校园、商品和头像演示图片，后端 uploads 目录保存用户上传文件。上传接口生成包含时间与随机字符串的文件名并返回资源路径。静态资源与业务数据分离能够避免将二进制图片写入关系表，但生产环境应使用对象存储和安全扫描。

4.3 系统功能模块设计

功能模块划分如图 4-2 所示。

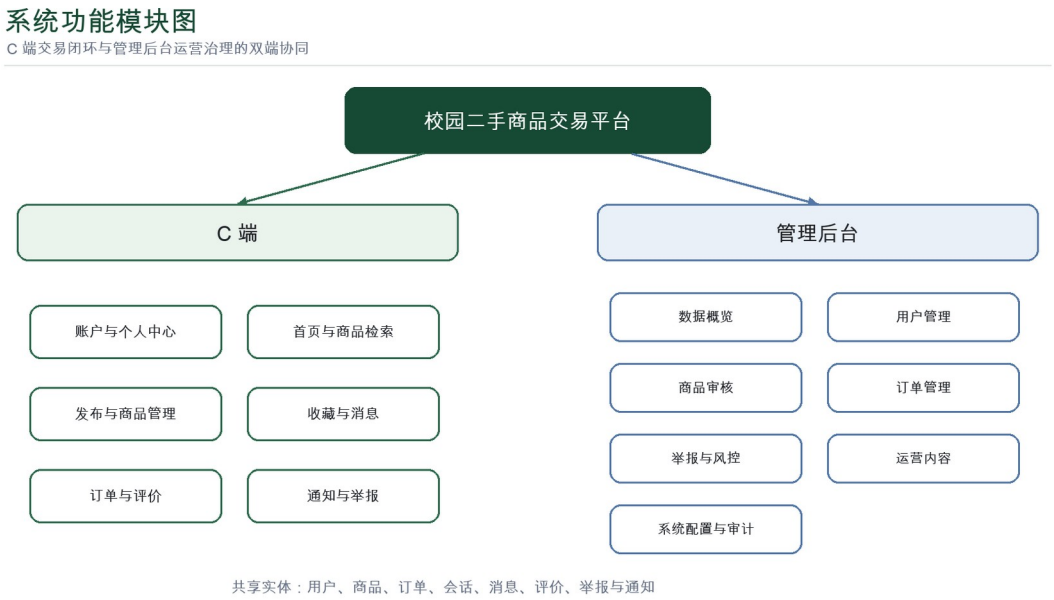


图 4-2 系统功能模块图

4.3.1 C 端功能模块设计

C 端分为账户与个人中心、首页与商品检索、发布与商品管理、收藏与消息、订单与评价、通知与举报六类模块。各模块不是孤立页面：商品详情连接收藏、会话、下单和举报；订单连接消息、状态与评价；个人中心连接发布与通知，从而形成以商品和用户关系为核心的交易链。

4.3.2 管理后台功能模块设计

管理后台分为数据概览、用户管理、商品审核、订单管理、举报与风控、运营内容、系统配置与审计七类模块。后台操作通过管理员接口完成，商品审核、用户状态和举报结果均写入审计日志，运营内容则影响 C 端分类、轮播或公告展示。

4.4 核心业务流程与状态设计

4.4.1 身份认证与权限流程

登录认证时序如图 4-3 所示。用户提交账号密码后，FastAPI 通过 ORM 查询用户，使用 PBKDF2 验证密码，账号有效时签发令牌。后续请求由 Axios 加入令牌，后端解析用户并根据接口要求检查登录或管理员角色。

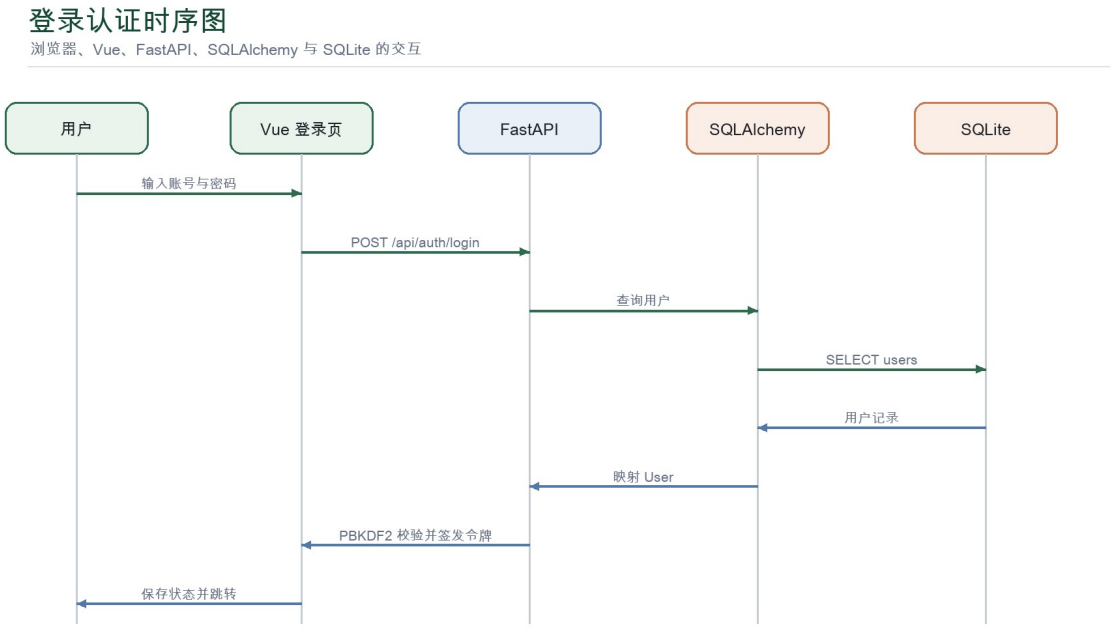


图 4-3 登录认证时序图

4.4.2 商品生命周期设计

普通用户发布商品时，根据配置进入 pending；管理员通过后进入 published，驳回进入 rejected。已发布商品可被用户或管理员下架为 off_shelf，管理员可恢复发布。用户修改商品时重新进入待审核。订单完成后商品进入 sold。前台公开列表只查询已发布商品，非公开状态仅资源所有者或管理员可查看。

4.4.3 订单状态机设计

订单初始状态为 pending，买家付款后为 paid，确认面交后为 meeting，买家完成后为 completed。取消进入 cancelled，退款和争议进入 refunding 或 disputed。不同动作由映射表转换状态，并验证操作者身份。商品与订单状态机如图 4-4 所示。

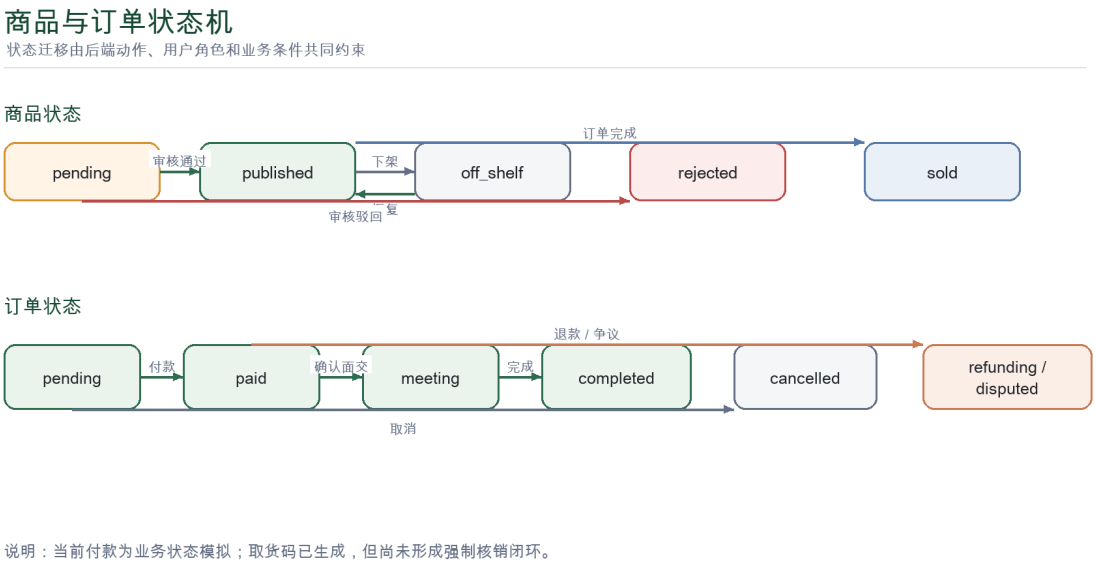


图 4-4 商品与订单状态机

4.4.4 会话与消息流程设计

会话以商品、买家和卖家三者组合建立唯一约束。买家从商品详情发起咨询时，后端查找已有会话或创建新会话；发送消息后更新会话的最后消息与时间。读取会话消息时，系统将非当前用户发送的消息标记为已读。该设计保证消息始终与交易商品关联。

4.4.5 举报处理状态设计

举报初始为 pending，管理员受理后可更新为 processing，完成后为 resolved。处理记录包含处理人、结果和处理时间。举报对象使用 target_type 与 target_id 表示，

可扩展到商品、用户和订单，但缺少强外键约束，因此业务层需要验证目标是否存在。

4.5 数据库设计

4.5.1 数据库概念结构设计

核心实体关系如图 4-5 所示。User 与 Product、Favorite、Order、Review、Report、Notification 和 AuditLog 发生关系；Category 对 Product 为一对多；Product 与 Order、Conversation 关联；Conversation 对 Message 为一对多；运营和配置实体由管理员维护。

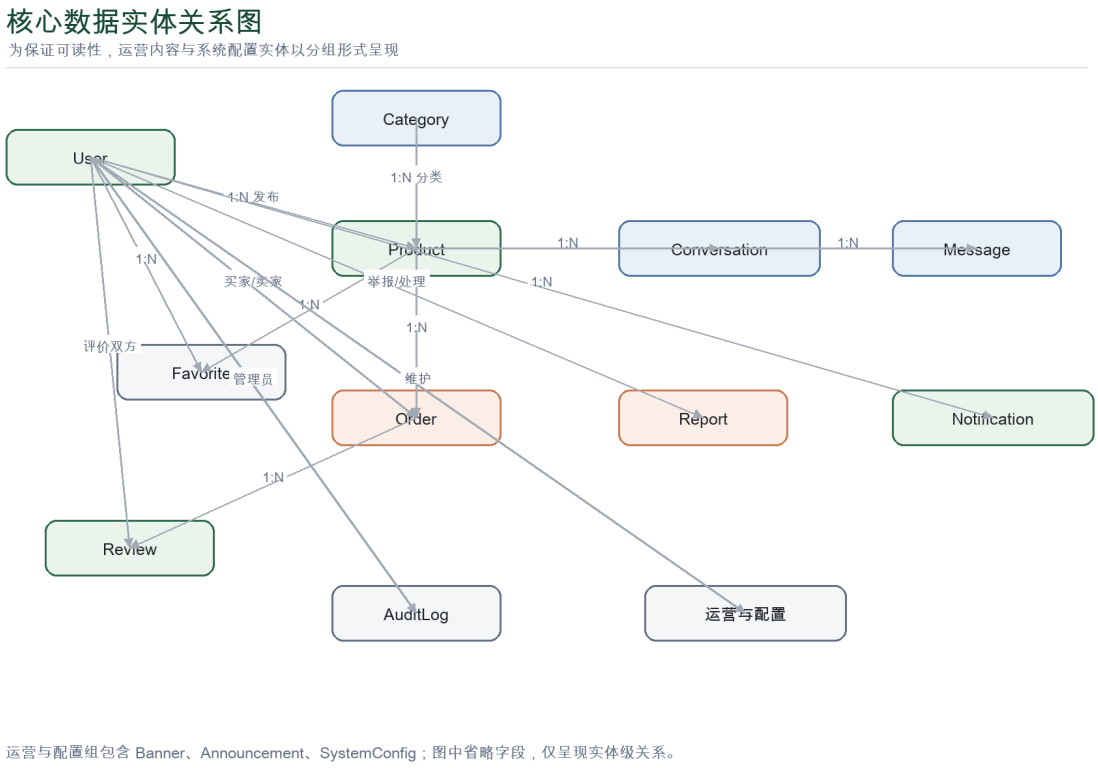


图 4-5 核心数据实体关系图

4.5.2 数据库逻辑结构设计

系统共设计 14 个主要实体，核心作用如表 4-1 所示。

表 4-1 核心数据实体说明

实体	核心字段	主要作用
User	username、role、campus、verified、status、credit_score	保存用户、角色、校园身份和信用状态
Category	name、icon、sort_order、enabled	定义商品分类与前台展示顺序
Product	title、price、condition、seller_id、status、images	保存商品内容、归属、图片与生命周期
Favorite	user_id、product_id	建立用户与商品的唯一收藏关系
Order	order_no、buyer_id、seller_id、status、pickup_code	记录交易主体、金额、状态和交接信息
Conversation	product_id、buyer_id、seller_id、last_message	以商品为上下文组织买卖会话
Message	conversation_id、sender_id、content、is_read	保存消息内容和已读状态
Review	order_id、reviewer_id、reviewee_id、score	保存订单完成后的双向评价
Report	target_type、target_id、status、handler_id、result	保存举报与管理员处理结果
Notification	user_id、title、type、is_read	保存订单、审核与系统通知
Banner	title、image、link、enabled	管理首页轮播内容
Announcement	title、content、pinned、enabled	管理公告与安全提醒
AuditLog	operator_id、action、target_type、detail	记录管理员关键操作

实体	核心字段	主要作用
SystemConfig	config_key、config_value、description	以键值方式保存系统参数

4.5.3 核心数据表设计

用户表使用用户名和邮箱唯一索引支持登录查找，角色与状态字段建立索引。商品表对标题、分类、卖家、状态和创建时间建立索引，便于市场筛选和后台审核。订单表使用唯一订单号并对商品、买卖双方和状态建立索引。会话表通过商品、买家、卖家联合唯一约束避免重复会话；收藏表通过用户与商品联合唯一约束避免重复收藏；评价表通过订单与评价人联合唯一约束限制每个用户对同一订单评价一次。

4.5.4 数据完整性与索引设计

外键用于表示实体引用，唯一约束用于避免重复关系，Pydantic 和业务逻辑用于限制字段范围和状态动作。当前 SQLite 配置满足原型运行，但仍需注意外键执行策略、并发写入和事务隔离。生产环境应为高频组合查询补充联合索引，并使用数据库事务保证单件商品只能被一个进行中订单占用。

4.6 REST API 设计

4.6.1 API 分组与统一约定

系统共包含 49 个路径操作，按认证与用户、商品与收藏、订单与评价、会话与消息、通知与举报、后台管理六组组织。接口统一返回 JSON，资源创建使用 201，权限和业务冲突使用明确状态码。分页接口返回 items、total、page 和 page_size。

表 4-2 REST API 分组

分组	代表接口	说明
认证与用户	POST /auth/login；GET /auth/me；PUT /profile	注册登录、当前用户和资料维护
商品与收藏	GET/POST /products；POST /products/{id}/favorite	商品检索、发布、修改、下架与收藏
订单与评价	POST /orders；PATCH /orders/{id}/status；POST /reviews	创建订单、状态动作和交易评价
会话与消息	POST/GET /conversations；GET/POST /messages	创建会话、读取与发送消息
通知与举报	GET /notifications；POST /reports	通知已读与异常上报
管理后台	/admin/dashboard、users、products、orders、reports 等	运营治理、配置与审计

4.6.2 认证与用户接口

注册接口检查账号与邮箱唯一性，登录接口支持多种账号字段，/auth/me 返回当前用户，/profile 更新允许修改的个人字段，/users/{id} 只返回公开资料与评价统计。私人响应与公开响应分开序列化，避免密码哈希等字段泄露。

4.6.3 商品与收藏接口

商品列表接口支持关键词、分类、成色、校区、价格和排序参数；详情接口返回商品和同类推荐；创建与修改接口使用同一 ProductPayload；删除接口通过修改状态实现逻辑下架。收藏切换接口根据关系是否存在执行新增或删除，并同步收藏数量。

4.6.4 订单与评价接口

创建订单接口验证商品可购买、买家不是卖家且同一买家没有进行中订单，然后生成订单号和取货码。状态接口根据动作映射目标状态并检查操作者。评价接口要求订单已完成，并通过唯一约束防止重复评价。

4.6.5 消息、通知与举报接口

会话接口确保商品买卖双方关系正确，消息接口限制访问者必须是会话参与者。通知接口按时间返回用户通知，并支持单条或全部已读。举报接口保存目标、原因和描述，管理接口负责筛选与处理。

4.6.6 管理后台接口

后台接口全部依赖 get_admin_user。数据概览接口执行聚合统计；用户状态、商品审核和举报处理接口同时调用 add_audit_log；运营接口管理分类、轮播和公告；设置接口批量保存配置；审计日志接口按时间倒序返回操作记录。

4.7 权限与安全设计

权限分为游客、登录用户和管理员三级。公开查询可使用可选用户依赖，以便在登录时附加收藏状态；受保护接口使用当前用户依赖；后台接口使用管理员依赖。资源级权限还要判断商品卖家、订单买卖双方和会话参与者。密码哈希、签名令牌、状态验证、上传扩展名限制和错误状态码构成当前安全基础。

4.8 前端界面与响应式设计

界面采用校园绿、象牙白与低饱和辅助色。C 端突出图片和价格，后台突出统计和表格。CSS 使用网格与弹性布局，在窄屏下减少商品列数、折叠导航和调整表

单。关键按钮使用明显主色，异常状态使用橙红色，状态文本同时保留文字，避免仅依赖颜色传达信息。

4.9 异常处理、配置与审计设计

后端通过 HTTPException 返回结构化错误，前端响应拦截器统一显示。系统配置使用键值表保存审核开关、图片数和服务费等参数，使部分规则无需修改代码。管理员修改用户、审核商品、处理举报和保存设置时写入审计日志，记录操作者、动作、对象、说明和时间。

4.10 本章小结

本章完成了系统架构、模块、认证、商品和订单状态、会话、举报、数据库、接口、安全与界面设计。设计结果将用户交易链和平台治理链连接起来，并为第 5 章的代码实现提供了明确结构。

第 5 章 系统详细实现

5.1 开发与运行环境

开发环境使用 Python 3.12 与 Node.js，后端依赖 FastAPI、SQLAlchemy、Pydantic、Uvicorn 和文件上传组件，前端使用 Vue 3、Vite、Vue Router、Pinia、Axios、Element Plus 与 ECharts。数据库文件位于 backend/campus_market.db，上传目录位于 backend/uploads，前端静态资源位于 frontend/public/assets。

表 5-1 开发技术与用途

层次	技术	用途
前端框架	Vue 3、Vite	组件化页面、响应式状态与构建
路由与状态	Vue Router、Pinia	页面导航、权限守卫和登录状态
请求与组件	Axios、Element Plus	REST 调用、表单、表格和提示
数据可视化	ECharts	后台趋势图与分类分布图
后端框架	FastAPI、Pydantic	API 路由、依赖注入与参数校验
数据访问	SQLAlchemy、SQLite	ORM 映射、查询、事务与本地存储
安全	PBKDF2、HMAC-SHA256	密码哈希和令牌签名

5.2 项目结构与启动流程

项目根目录分为 backend、frontend 和 scripts。后端 app 目录包含数据库、模型、安全、序列化、种子数据和主路由；前端 src 目录包含请求封装、公共组件、布

局、路由、状态和视图；scripts/dev.sh 用于一键启动前后端。应用启动时后端创建表并写入演示数据，前端开发服务器启动后通过代理访问 8000 端口。

后端请求处理过程如图 5-1 所示。请求经过路由、认证与参数校验后执行业务规则和 ORM 操作；认证、参数或资源异常会以相应状态码提前返回。

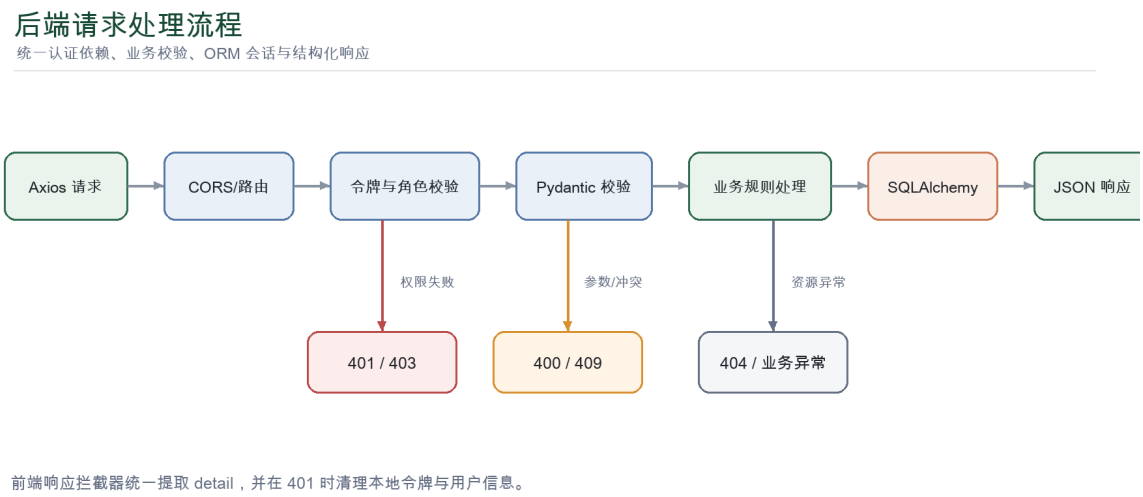


图 5-1 后端请求处理流程

5.3 用户注册登录与权限控制实现

注册接口使用 RegisterPayload 限制账号、邮箱、密码与昵称长度，在数据库查询重复账号或邮箱。创建用户前调用 hash_password 生成盐和派生值。登录接口从用户名、邮箱或手机号中匹配用户，验证密码和账号状态，更新最后登录时间并返回令牌。前端 Pinia Store 的 persist 方法保存令牌与用户；应用刷新时调用 /auth/me 重新获取用户状态。

Vue Router 的全局守卫检查路由元信息：未登录访问发布、收藏、消息、订单或个人中心时跳转登录；非管理员访问 /admin 时跳转登录；已登录用户访问登录注册页时返回对应首页。服务端再次验证令牌与角色，因此即使绕过前端路由，也不能调用无权接口。

5.4 首页推荐与商品市场实现

首页接口一次返回轮播、公告、分类和推荐商品。商品查询仅选择 published，按推荐与创建时间排序；登录用户额外查询收藏关系，以便卡片显示收藏状态。首页真实页面如图 5-2 所示，包含校园主题区、分类入口、同校好物、安全提示和平台特色。



图 5-2 C 端首页

市场接口将关键词、分类、成色、校区、价格范围转化为 SQLAlchemy 条件，根据 latest、price_asc、price_desc、popular 或 recommended 选择排序，并使用 offset 和 limit 分页。前端仅提交非空筛选参数，结果变化后更新列表与总数。

5.5 商品详情、收藏与举报实现

商品详情接口加载卖家和分类，非公开状态只允许卖家或管理员查看，同时增加浏览量并查询同分类推荐。收藏接口查询当前关系，存在则删除并减少收藏数，不存在则新增并增加收藏数。联合唯一约束从数据库层防止重复收藏。

详情页展示商品图片、价格、成色、校区、取货地点、交易方式、卖家认证与信用信息，并提供收藏、联系卖家、立即购买和举报操作。下单确认弹窗要求买家确认见面地点和备注，界面如图 5-3 所示。



图 5-3 商品详情与下单确认界面

5.6 商品发布、编辑与审核实现

发布页分为商品图片、基本信息、价格与成色、交易信息和真实性承诺。前端限制最多 9 张图片，上传后将接口返回路径加入表单；提交时根据是否存在路由参数调用创建或修改接口。发布页面如图 5-4 所示。

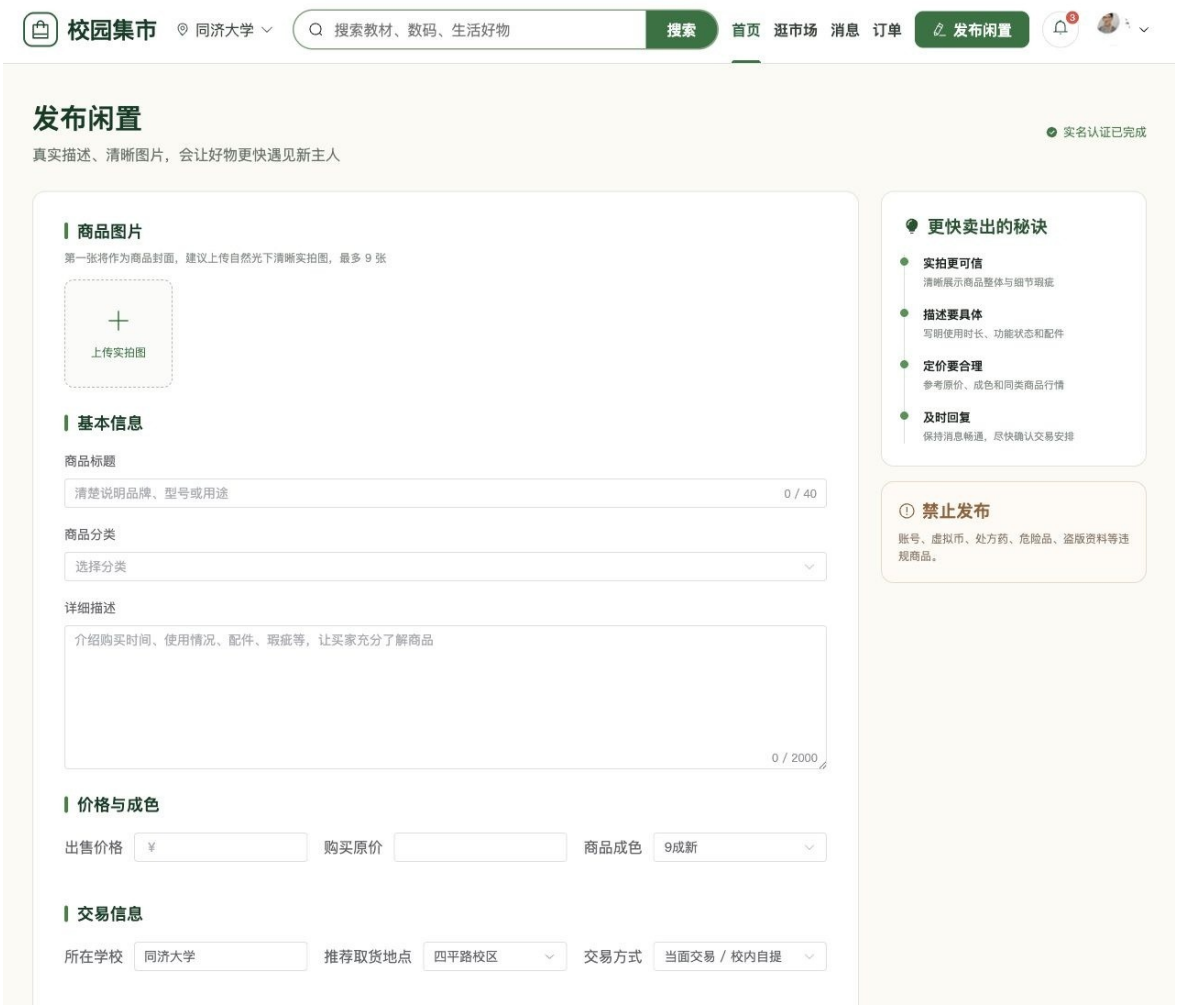


图 5-4 商品发布界面

后端 ProductPayload 对标题、描述、价格和图片数量进行校验。普通用户发布时根据审核配置进入待审核，管理员审核后改变状态并填写审核说明。用户编辑商品后重新待审，从流程上防止发布后通过编辑绕过内容审核。

5.7 商品会话与消息中心实现

用户点击联系卖家时，前端提交商品编号。后端确认商品存在且当前用户不是卖家，然后根据商品、买家和卖家查找会话，没有则创建。消息中心先加载会话列表，选择会话后加载消息；发送消息时写入 Message，并同步 Conversation 的最后

消息与时间。消息列表按时间排序，访问控制确保只有会话参与者可以读取或发送。

当前消息基于 REST 请求，不具备真正实时推送、输入状态或消息撤回。论文将其定义为“商品上下文消息”，而不是 WebSocket 即时通信。后续可以在保留会话与消息表的基础上增加 WebSocket 或 SSE。

5.8 订单创建与状态流转实现

创建订单时，后端验证商品状态、禁止购买自己的商品，并检查当前买家是否已有进行中订单。系统使用时间和随机数生成订单号，使用 6 位随机数生成取货码，同时为卖家创建新订单通知。订单状态接口根据动作映射为目标状态，校验买家或参与者权限，并记录付款、完成时间与取消原因。

订单页提供全部、待付款、已付款、待面交、已完成和售后/争议筛选，并可区分买入和卖出。每张订单卡显示商品、交易对象、地点、取货码、金额、状态节点与当前可执行操作，真实页面如图 5-5 所示。

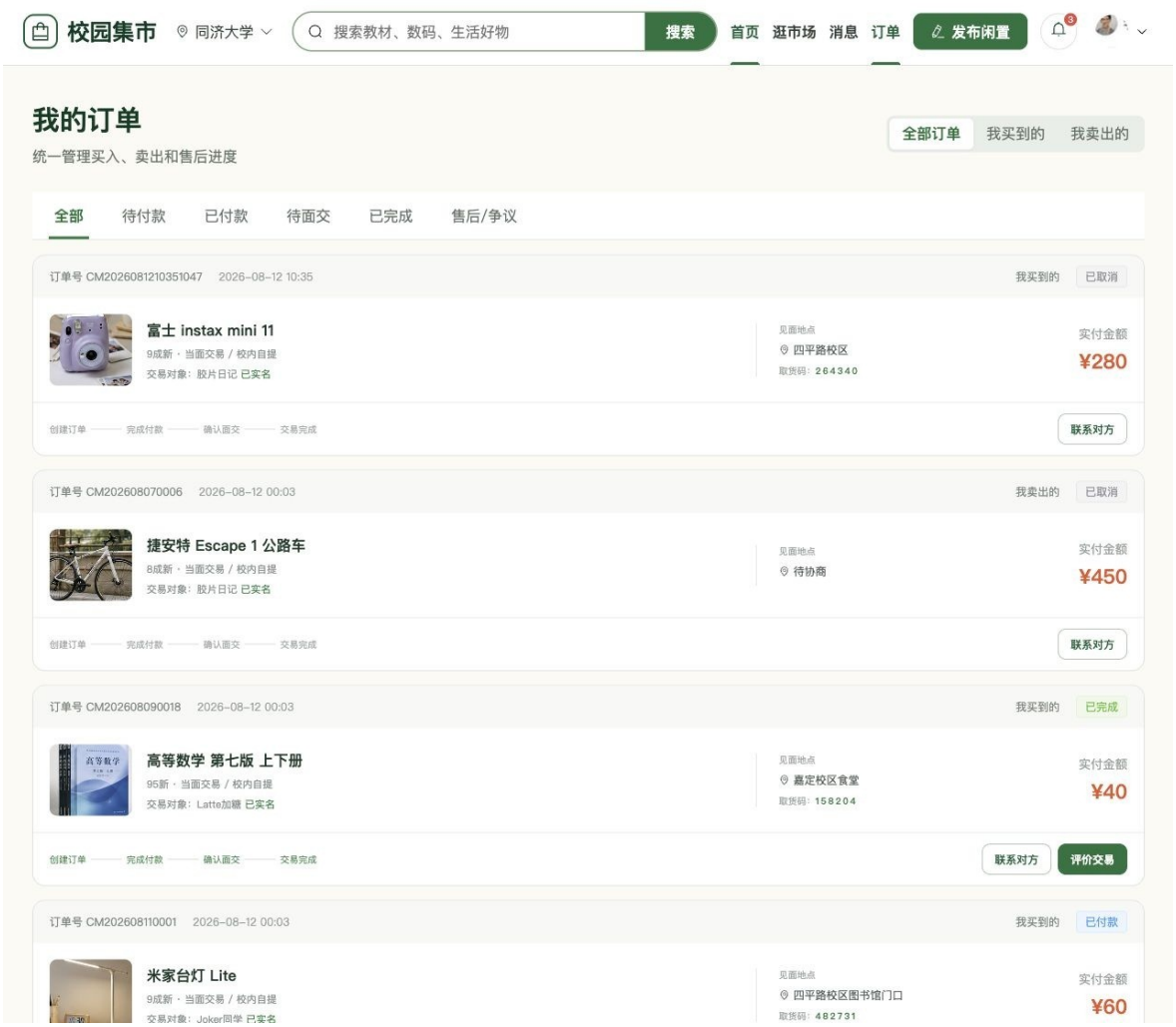


图 5-5 C 端订单管理界面

当前订单实现仍存在单品并发占用风险：逻辑只阻止同一买家重复创建进行中订单，没有从数据库层阻止多个买家同时占用同一商品。真实上线应增加 available/reserved/sold 库存状态和事务锁，并通过卖家接单或超时释放实现唯一占用。

5.9 评价、信用与通知实现

订单完成后，参与交易的用户可以提交 1～5 分评价。后端判断订单状态与评价者身份，设置被评价人，并依赖订单和评价者联合唯一约束防止重复。用户详情接

口聚合收到的评价数量与平均分；个人中心展示信用分、认证、交易和评价信息，如图 5-6 所示。



图 5-6 个人信用与认证界面

通知由订单和举报等业务创建，按时间倒序返回，支持单条和全部已读。当前信用分主要来自种子数据和展示字段，没有实现复杂增减规则，因此不能将其描述为成熟信用风控模型。

5.10 管理后台数据概览实现

后台概览接口通过聚合查询得到用户、商品、订单、交易额、待审核商品、进行中订单和待处理举报；按日期聚合近 7 日订单数与金额；按分类统计商品数量；最后返回最新订单。前端使用统计卡片、折线柱状组合图、环形图和订单表展示数据，页面如图 5-7 所示。

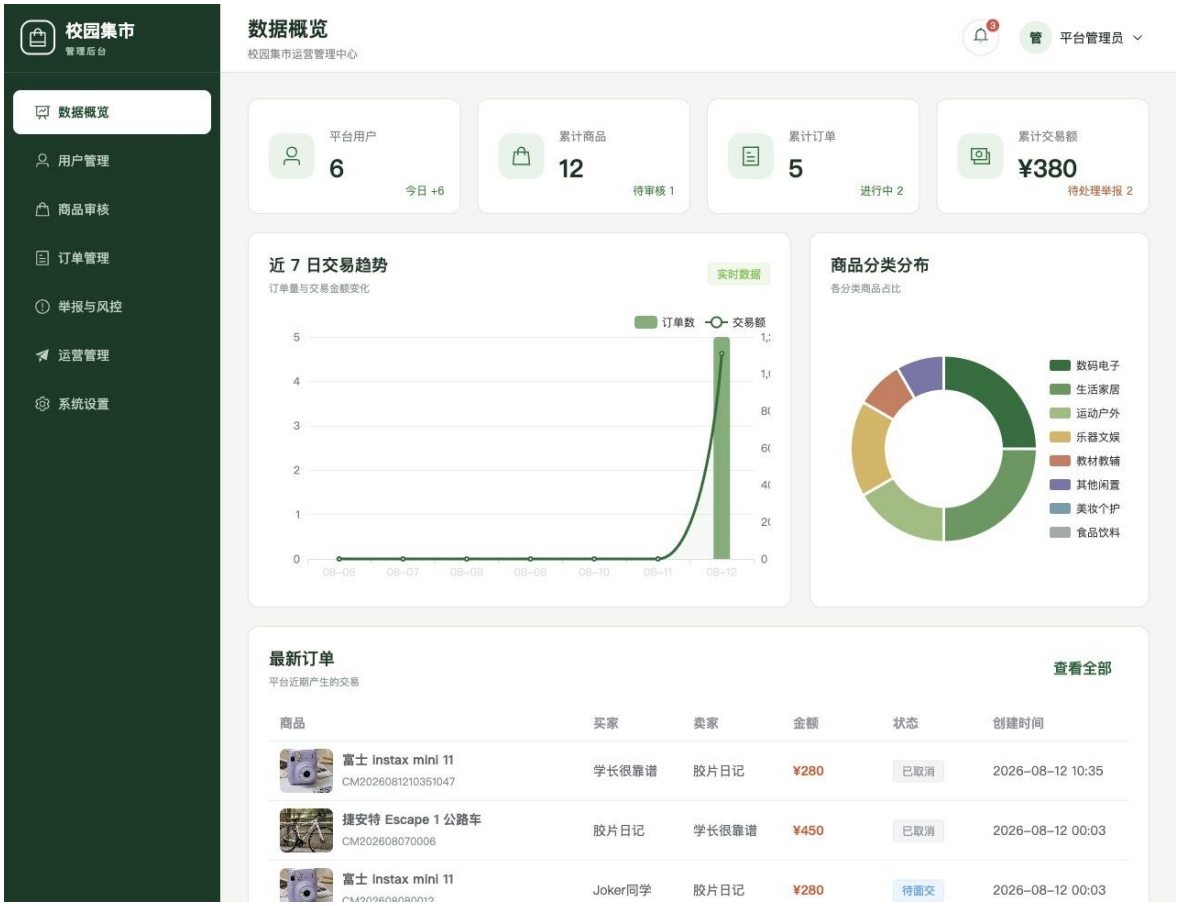


图 5-7 管理后台数据概览

5.11 管理后台业务管理实现

5.11.1 用户状态管理实现

用户列表支持关键词、状态和角色筛选。管理员选择暂停、恢复或封禁并填写原因，后端更新状态且写入审计日志。认证依赖在每次请求时检查用户必须为 active，因此账号停用后已有令牌也不能继续访问受保护接口。

5.11.2 商品审核管理实现

商品管理支持关键词、状态和分类筛选，管理员可审核通过、驳回、下架、恢复、推荐和取消推荐。审核动作使用映射表更新状态，推荐动作修改 featured，所有动作写入日志。待审数量同时反映在概览指标中。

5.11.3 订单查询管理实现

订单后台支持按订单号、商品或用户检索，并按状态筛选。列表展示商品、买家、卖家、金额、状态和创建时间，用于运营查询。当前后台为只读查询，没有实现冻结、退款裁决或完整工单时间线。

5.11.4 举报处理实现

举报后台按状态与对象类型筛选，管理员填写处理状态和结论。解决举报时记录处理人和时间，并向举报人创建通知，同时写入审计日志。真实页面如图 5-8 所示。

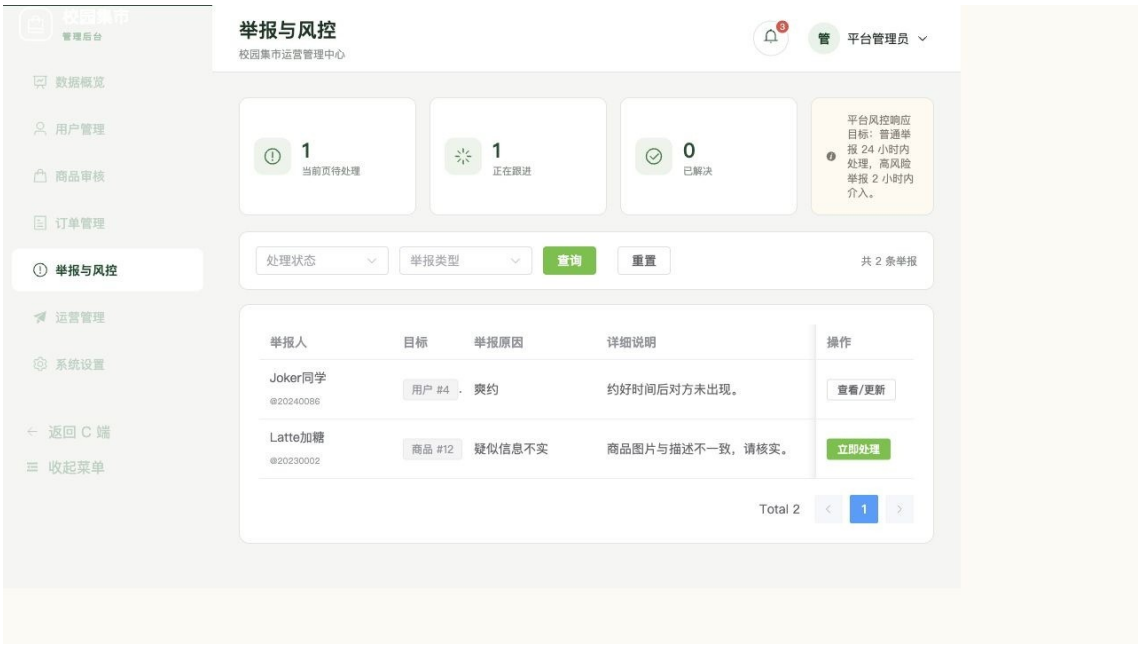


图 5-8 管理后台举报与风控界面

5.11.5 分类、轮播与公告管理实现

运营管理页面并行加载分类、轮播和公告。分类支持新增、修改、排序和启用；轮播保存标题、副标题、图片、链接与顺序；公告保存类型、置顶和启用状态。保存后重新加载数据，使运营人员无需修改前端代码即可调整首页内容。

5.11.6 系统配置与审计日志实现

设置接口读取所有配置并以键为索引返回。前端编辑审核开关、最大图片数、服务费等值后批量保存。后端对已存在配置更新值，对不存在配置创建记录，并写入管理员日志。审计日志按时间倒序展示操作者、动作、对象与说明。

5.12 文件上传与安全机制实现

上传接口要求登录，并限制文件扩展名为 JPG、JPEG、PNG、WEBP 或 GIF。文件名由当前时间和随机令牌组成，降低重名覆盖风险。认证方面，密码使用加盐 PBKDF2，令牌使用 HMAC 签名并设置过期时间。接口通过依赖检查用户、角色和状态。

当前文件验证只检查扩展名，仍需增加实际 MIME、文件大小、图片解码、像素限制、EXIF 清理和违规内容检测；默认 APP_SECRET 仅适合本地开发；访问令牌没有刷新与撤销列表。这些问题在真实部署前必须补齐。

5.13 响应式布局与交互体验实现

系统使用 CSS Grid 和 Flexbox 组织商品网格、页面头部、表单和后台内容，并针对窄屏调整列数、间距、导航和按钮排列。图片设置自适应宽度和对象裁剪，防止卡片比例失衡。页面统一提供加载、空状态、确认弹窗和成功或错误消息，降低操作不确定性。

5.14 本章小结

本章结合代码与真实界面说明了登录权限、首页市场、详情收藏举报、发布审核、消息、订单、评价通知和后台运营等模块实现。系统已形成可运行的双端原型，同时明确了库存锁定、实时消息、信用规则和文件安全等边界。

第 6 章 系统测试

6.1 测试目标与测试策略

测试目标是验证系统是否满足第 3 章需求，并确认关键页面、接口、权限和状态转换能够正常工作。测试采用静态检查、构建测试、黑盒功能测试、接口响应测试、权限测试和浏览器页面检查相结合的方式。涉及真实支付、权威学籍认证、渗透攻击与生产压力的内容不在当前测试范围。

6.2 测试环境与工具

测试日期为 2026 年 8 月 12 日，本地系统运行于 macOS 开发环境。前端通过 Vite 开发服务器运行在 127.0.0.1:5173，后端通过 Uvicorn 运行在 127.0.0.1:8000，数据库为 SQLite。前端使用 npm run build 验证生产构建，后端使用 Python py_compile 验证模块语法；页面通过浏览器桌面和移动宽度检查；只读接口测试使用自编脚本以并发数 5 重复请求。

6.3 功能测试

选取覆盖主业务链的测试用例如表 6-1 所示。

表 6-1 核心功能测试用例

编号	测试功能	操作与预期结果	结果
F01	用户登录	使用演示用户账号登录，进入 C 端并显示用户导航	通过
F02	商品筛选	输入关键词并选择分类/价格，返回匹配商品与总数	通过
F03	商品发布	填写完整表单提交，普通用户商品进入待审核	通过

编号	测试功能	操作与预期结果	结果
F04	商品收藏	点击收藏后状态和收藏数量更新，再次点击取消	通过
F05	商品会话	从详情联系卖家，进入以商品为上下文的会话	通过
F06	创建订单	非卖家用户确认地点后创建订单并生成订单号	通过
F07	订单状态	按权限执行付款、面交、完成或取消，状态更新	通过
F08	交易评价	已完成订单提交一次评价，重复提交被阻止	通过
F09	商品审核	管理员审核通过/驳回并记录说明	通过
F10	举报处理	管理员更新举报状态和结论，用户收到通知	通过
F11	运营管理	新增或修改分类、轮播和公告后列表刷新	通过
F12	系统设置	保存配置并在审计日志中出现操作记录	通过

测试表明主要功能能够覆盖商品信息流、交易流和运营治理流。由于测试使用演示数据，结论只表示当前版本在指定操作路径下工作正常，不代表所有异常输入和长期运行均无缺陷。

6.4 API 与权限测试

接口权限测试重点验证未登录、普通用户和管理员三种身份。未登录访问 /api/orders 返回 401；普通用户访问 /api/admin/dashboard 返回 403；无关用户修改他人商品或访问非参与会话返回 403；不存在资源返回 404；重复进行中订单返回 409。前端收到 401 后清理本地令牌，路由守卫将用户引导到登录页。

后端共有 49 个路径操作，FastAPI 能够自动生成 OpenAPI 接口文档。请求模型对标题、描述、价格、评分和状态动作进行约束，减少非法数据进入业务层。仍需补充自动化 pytest 测试套件和独立测试数据库，以提高回归测试覆盖率。

6.5 性能与并发测试

为了获得可复现的基础数据，本文选择 /api/health、/api/home、/api/products 和 /api/categories 四个只读接口，每个接口请求 30 次，并发数为 5，共执行 120 次请求。测试均在本机 localhost 环境完成，结果如表 6-2 与图 6-1 所示。

表 6-2 本地只读接口响应测试结果

接口	请求数	成功数	平均响应/ms	中位数/ms	P95/ms
/api/health	30	30	5.04	1.34	23.77
/api/home	30	30	15.26	15.05	20.09
/api/products	30	30	12.57	12.88	14.62
/api/categories	30	30	4.01	4.03	4.79

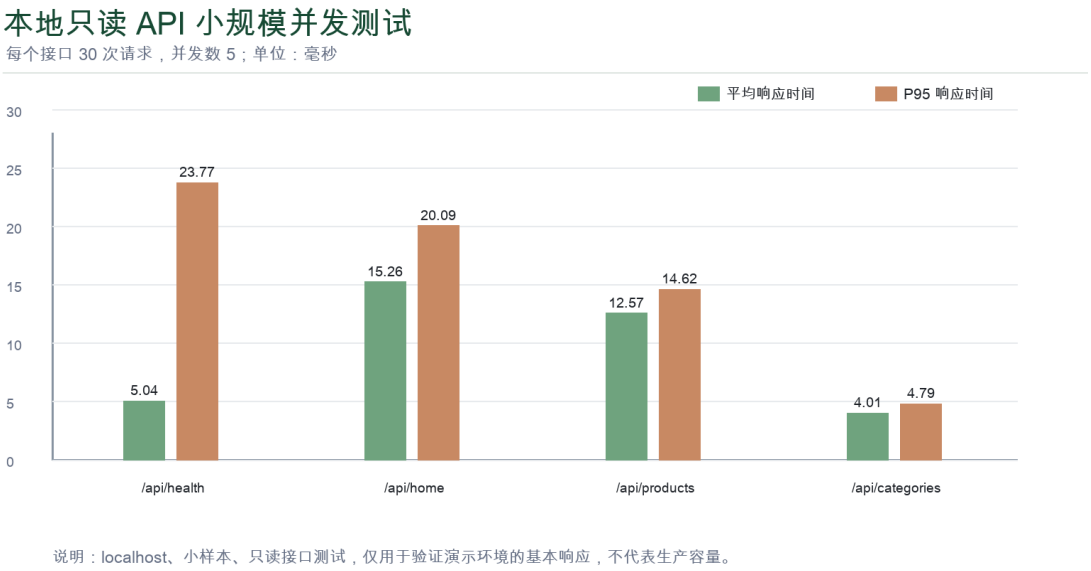


图 6-1 本地只读 API 小规模并发测试

120 次请求全部成功，平均响应时间为 4.01～15.26 ms。首页接口返回轮播、公告、分类和商品，响应体较大，因此平均耗时高于健康检查和分类接口。健康检查的平均值受少量首次连接耗时影响，P95 高于中位数。该测试样本小、网络距离为零、数据量有限且没有写请求，不能用于声明系统满足高并发或生产 SLA。SQLite 写并发、商品库存锁定和真实部署网络仍需单独测试。

6.6 安全性测试

安全检查包括密码存储、令牌篡改、令牌过期、角色越权、资源越权、账号停用和上传扩展名。数据库中不保存明文密码；修改令牌任意部分后签名验证失败；过期令牌无法解析；普通用户无法调用管理接口；非卖家不能修改商品；非会话参与者不能读取消息；被停用用户即使持有旧令牌也不能通过当前用户依赖；非允许扩展名上传返回 400。

当前安全测试不是渗透测试。系统尚未实现验证码、登录限流、CSRF 专项防护、刷新令牌与撤销、文件内容扫描、隐私数据加密、审计告警和安全响应流程。原型使用本地默认密钥，真实部署前必须通过环境变量替换并启用 HTTPS。

6.7 兼容性与响应式测试

浏览器检查覆盖首页、市场、登录、消息、订单、发布和管理后台。桌面宽度下商品网格、顶部导航、后台侧栏和图表显示正常；移动宽度下 C 端页面能够调整列数和导航，不出现整页横向溢出。后台主要面向桌面运营，窄屏下表格密度仍是后续优化重点。

前端生产构建成功，共转换 2260 个模块。构建输出提示主入口与后台图表分块体积超过 500 kB，这不影响功能通过，但表明 ECharts 和 UI 组件存在进一步按需加载与分包优化空间。

6.8 测试结果与缺陷分析

综合测试说明主要功能、权限、构建和本地只读响应满足演示要求。发现的关键缺陷包括：校园认证只依据学号字段改变状态；多个买家可能同时为单件商品创建订单；取货码没有强制核销；付款仅为状态模拟；消息为 REST 拉取；售后争议缺少独立工单；上传缺少内容级校验；自动化测试不足；前端构建块较大。

这些缺陷不影响当前作为毕业设计原型展示完整流程，但会影响真实上线。后续应优先实现认证审核、库存事务、取货码核销、售后工单和安全加固，再开展服务器型数据库下的并发、渗透和长期稳定性测试。

6.9 本章小结

本章通过功能用例、权限检查、构建检查、浏览器验证和本地只读接口测试对系统进行了评价。测试结果证明原型能够运行并完成主要业务，但同时确认其不具备直接生产上线条件，测试结论的适用范围得到明确限制。

第 7 章 总结与展望

7.1 研究工作总结

本文围绕校园闲置交易信息分散、检索困难、过程缺少留痕和平台治理薄弱等问题，设计并实现了基于 FastAPI 与 Vue 的校园二手商品交易平台。系统采用前后端分离架构，完成 C 端与管理后台，建立 14 类核心实体和 49 个 API，覆盖商品发布审核、浏览筛选、收藏、商品会话、订单状态、评价通知、举报风控、运营内容、系统配置和审计日志。

论文完成了从研究背景、需求、架构、模块、状态、数据库和接口到详细实现与测试的完整论证。真实页面截图表明双端界面已经实现，构建和语法检查能够通过，本地只读接口测试 120 次请求全部成功。系统将学生交易和管理员治理连接为可追溯链路，达到了毕业设计原型的预期目标。

7.2 系统不足

系统仍存在明显边界。第一，校园认证为演示级，没有连接教育邮箱或学籍系统。第二，订单缺少商品唯一占用事务和卖家接单，存在多买家竞争风险。第三，付款和取货码是业务状态模拟，没有真实资金、核销和账务。第四，消息没有 WebSocket 实时推送。第五，退款争议没有独立工单、证据和裁决。第六，SQLite、默认密钥与基础上传检查不适合直接生产。第七，性能样本有限，自动化测试和可访问性验证不足。

7.3 未来展望

后续工作首先应完善校园邮箱验证码、材料审核和管理员认证工作台；为商品增加 available、reserved、sold 状态，使用 PostgreSQL/MySQL 事务和唯一约束保证

单件占用；增加卖家接单、改约、取货码核销、超时释放和售后工单。消息模块可引入 WebSocket 或 SSE，支持已读回执、输入状态和图片位置消息。

在运营和安全方面，可增加敏感词、高风险商品、设备与 IP 风险、黑名单、申诉和审计告警；将图片迁移到对象存储并进行转码、EXIF 清理和内容检测；采用 HTTPS、刷新令牌、会话撤销、限流和密钥轮换。在校园特色方面，可扩展教材课程与 ISBN、求购、毕业季专题、校内交接点地图、议价、降价提醒和闲置流转环保成就，使平台从交易原型发展为校园资源循环服务。

参考文献

[1] 教育部. 绿色低碳发展国民教育体系建设实施方案[EB/OL]. 2022-11-08. https://www.moe.gov.cn/srcsite/A03/moe_1892/moe_630/202211/t20221108_979321.html.

[2] 季悦, 彭欣悦, 方媛媛, 等. 校园二手交易平台存在的问题与对策研究——以甘肃农业大学为例[J]. 服务科学和管理, 2025, 14(5): 677-688. DOI:10.12677/ssem.2025.145083.

[3] Wei S J, Fang D, Mo M, et al. Research on the Practice of College Students' Second Hand Trading Platform[J]. Open Access Library Journal, 2023, 10: 1-7. DOI:10.4236/oalib.1110405.

[4] 杨娇. 大学校园二手商品交易市场存在的问题及对策[J]. 科技创业月刊, 2018, 31(2): 57-60. DOI:10.3969/j.issn.1672-2272.2018.02.016.

[5] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures[D]. University of California, Irvine, 2000.

[6] 张子欣, 高敏, 王斯宇, 等. 校园内部二手交易平台研究[J]. 合作经济与科技, 2020(9): 84-87.

[7] 曹振兴, 刘亚娇, 邹敏, 等. 绿色校园背景下的校园二手交易平台[J]. 数字通信世界, 2023(1): 75-77.

[8] 孙丽, 王皓, 戴璐, 等. 大学校园二手交易平台构建与运营——以 E 大学“花梨闲转”微信小程序为例[J]. 科技与创新, 2024(4): 12-16.

[9] Vue.js Team. Vue.js Guide: Introduction[EB/OL]. https://vuejs.org/guide/introduction.html.

[10] Vue.js Team. Vue Router Documentation: Introduction[EB/OL]. https://router.vuejs.org/introduction.

[11] Vue.js Team. Pinia Documentation: Introduction[EB/OL]. https://pinia.vuejs.org/introduction.html.

[12] FastAPI. FastAPI Features[EB/OL]. https://fastapi.tiangolo.com/features/.

[13] SQLAlchemy Authors. SQLAlchemy 2.0 ORM Documentation[EB/OL]. https://docs.sqlalchemy.org/en/20/orm/.

[14] SQLite Consortium. Features of SQLite[EB/OL]. https://sqlite.org/features.html.

[15] SQLite Consortium. SQLite Is Serverless[EB/OL]. https://www.sqlite.org/serverless.html.

[16] Python Software Foundation. hashlib — Secure hashes and message digests[EB/OL]. https://docs.python.org/3/library/hashlib.html.

[17] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT)[S]. RFC 7519, 2015.

[18] OWASP Foundation. Password Storage Cheat Sheet[EB/OL]. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.

[19] OWASP Foundation. REST Security Cheat Sheet[EB/OL]. https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html.

[20] Apache ECharts. ECharts Documentation[EB/OL]. https://echarts.apache.org/.

致谢

本论文从选题、需求分析、系统设计到编码测试的完成，离不开课程学习和软件工程方法的积累。感谢指导教师在选题范围、系统结构和论文规范方面给予的指导，帮助本文将功能开发整理为完整的研究过程。感谢同学在校园交易场景、页面交互和测试体验方面提出建议，使系统能够更加贴近学生实际使用需求。

同时感谢 Python、FastAPI、Vue、SQLAlchemy、SQLite、Element Plus 和 ECharts 等开源项目及其维护者提供的框架与文档。最后感谢家人和朋友在开发与写作期间给予的理解与支持。由于个人能力和时间有限，系统仍存在认证、并发、支付、售后和安全等不足，恳请各位教师批评指正。

附录 A 主要 API 清单

表 A-1 系统 API 路径操作清单

序号	方法与路径	功能
1	GET /api/health	健康检查
2-4	POST /api/auth/register ; POST /api/auth/login ; GET /api/auth/me	注册、登录和当前用户
5-7	PUT /api/profile ; GET /api/users/{id} ; POST /api/upload	资料、公开用户和上传
8-16	categories 、 home 、 products 、 favorites 、 my/ products	分类、首页、商品与收藏
17-20	orders 、 orders/{id} 、 orders/{id}/status、reviews	订单与评价
21-24	conversations 、 messages	会话与消息
25-28	notifications、reports	通知与举报
29-49	/api/admin/*	概览、用户、商品、订单、举报、运营、设置与审计

附录 B 测试数据说明

系统首次启动自动创建表并生成演示用户、商品、订单、消息、通知、举报、轮播、公告和配置数据。C 端演示账号为 20230001，管理员演示账号为 admin。演示数据仅用于功能展示和论文截图，不应在生产环境保留默认密码。